

Contents

1	Implementation Notes for Perspective Transformation	3
1.1	Table of Contents	3
1.2	Introduction	3
1.2.1	Purpose	3
1.2.2	Codebase Overview	3
1.3	Homography Solver Implementation	4
1.3.1	Algorithm Overview: Direct Linear Transformation (DLT)	4
1.3.2	Implementation: <code>compute_homography()</code>	4
1.3.3	Step 1: Input Validation	4
1.3.4	Step 2: Build Constraint Matrix A	5
1.3.5	Step 3: Solve Using SVD	6
1.3.6	Step 4: Reshape and Normalize	6
1.3.7	Complete Implementation	7
1.3.8	Testing Strategy	7
1.3.9	Performance Considerations	9
1.4	IPM Class Design	9
1.4.1	Architecture Overview	9
1.4.2	Class Structure	9
1.4.3	Initialization	10
1.4.4	Computing Source Points	11
1.4.5	Computing Destination Points	12
1.4.6	Image Transformation	13
1.4.7	Inverse Image Transformation	13
1.4.8	Point Transformation	14
1.4.9	Design Decisions	15
1.5	OpenCV Integration	16
1.5.1	<code>cv2.warpPerspective()</code> Deep Dive	16
1.5.2	Interpolation Flags	16
1.5.3	Border Modes	17
1.5.4	<code>cv2.getPerspectiveTransform()</code> vs. Custom DLT	17
1.5.5	Performance Comparison	18
1.6	Interactive Calibration Tool	18
1.6.1	Design Overview	18
1.6.2	Implementation: InteractiveIPM Class	19
1.6.3	Mouse Callback Handler	19
1.6.4	Display Update	20
1.6.5	BEV Computation and Visualization	21
1.6.6	Main Loop	22
1.6.7	Usage Example	23
1.7	Calibration Persistence	24
1.7.1	File Format: YAML	24
1.7.2	Save Calibration	25
1.7.3	Load Calibration	25
1.7.4	Usage Example	26
1.7.5	Multiple Calibration Profiles	26
1.8	Point Selection Strategy	27

1.8.1	Manual Selection Guidelines	27
1.8.2	Adaptive Point Selection	27
1.8.3	Destination Rectangle Sizing	28
1.9	Coordinate Transformation Details	29
1.9.1	Homogeneous Coordinate Conversion	29
1.9.2	Batch Transformation Efficiency	29
1.9.3	Handling Points Outside Valid Region	30
1.9.4	Numerical Precision Considerations	30
1.9.5	Round-Trip Transformation Validation	31
1.10	Validation and Testing	31
1.10.1	Unit Testing Strategy	31
1.10.2	Integration Testing	33
1.10.3	Accuracy Metrics	33
1.10.4	Validation Report	35
1.11	Performance Optimization	36
1.11.1	Pre-computation Strategies	36
1.11.2	Image Size Trade-offs	36
1.11.3	Interpolation Method Selection	37
1.11.4	Memory Usage Considerations	37
1.11.5	Profiling Results	37
1.11.6	GPU Acceleration (Optional)	38
1.12	Common Pitfalls and Debugging	39
1.12.1	Pitfall 1: Point Correspondence Order Mismatch	39
1.12.2	Pitfall 2: Coordinate System Confusion	39
1.12.3	Pitfall 3: Collinear Points	39
1.12.4	Pitfall 4: Numerical Instability	40
1.12.5	Pitfall 5: Forgetting to Normalize Homography	41
1.12.6	Pitfall 6: Incorrect Homogeneous Division	41
1.12.7	Debugging Checklist	41
1.13	Failure Modes and Limitations	42
1.13.1	When IPM Fails	42
1.13.2	Visual Artifacts	43
1.13.3	Mitigation Strategies Summary	43
1.14	Best Practices	44
1.14.1	Calibration Workflow Recommendations	44
1.14.2	When to Recalibrate	44
1.14.3	Multi-Camera Considerations	44
1.14.4	Integration with Lane Detection Pipeline	45
1.14.5	Production Deployment Considerations	45
1.15	Extension Points	46
1.15.1	Metric BEV Generation	46
1.15.2	Multi-Plane Transformations	47
1.15.3	Integration with Deep Learning	47
1.15.4	Real-Time Video Processing	48
1.16	References and Resources	49
1.16.1	Key Papers on IPM	49
1.16.2	OpenCV Documentation	49
1.16.3	Related Codebases	49

1 Implementation Notes for Perspective Transformation

=> **Practical implementation guide for homography-based IPM and Bird's Eye View transformation**

This document provides detailed implementation guidance, design decisions, best practices, and debugging strategies for building a robust Inverse Perspective Mapping (IPM) system.

1.1 Table of Contents

1. Introduction
2. Homography Solver Implementation
3. IPM Class Design
4. OpenCV Integration
5. Interactive Calibration Tool
6. Calibration Persistence
7. Point Selection Strategy
8. Coordinate Transformation Details
9. Validation and Testing
10. Performance Optimization
11. Common Pitfalls and Debugging
12. Failure Modes and Limitations
13. Best Practices
14. Extension Points
15. References and Resources

1.2 Introduction

1.2.1 Purpose

This document bridges the gap between mathematical theory (see `math_foundations.md`) and practical implementation. It covers: - **Architecture:** How to structure your IPM codebase - **Implementation:** Detailed coding guidance with examples - **Optimization:** Performance considerations and profiling - **Debugging:** Common issues and solutions - **Production:** Deployment best practices

1.2.2 Codebase Overview

The project is organized into modular components:

```
src/
homography.py      # DLT algorithm, homography computation
ipm.py             # IPM transformation class
interactive_tool.py # GUI calibration tool
visualizer.py      # Visualization utilities
```

Design Philosophy: - **Modularity:** Each component has a single responsibility - **Testability:** Functions are pure and testable - **Clarity:** Readability over cleverness - **Performance:** Optimize after correctness

1.3 Homography Solver Implementation

1.3.1 Algorithm Overview: Direct Linear Transformation (DLT)

The DLT algorithm computes the homography matrix $\mathbf{H}$ from point correspondences.

Key Steps: 1. Validate input (e4 points, non-collinear) 2. Build constraint matrix $\mathbf{A}$ from correspondences 3. Solve using SVD: $\mathbf{A} = \mathbf{U} \rightarrow \rightarrow \rightarrow \mathbf{V}^T$ 4. Extract solution from last column of $\mathbf{V}$ 5. Reshape to $3 \rightarrow 3$ matrix 6. Normalize (typically $\mathbf{H}[2,2] = 1$)

1.3.2 Implementation: `compute_homography()`

Function Signature:

```
def compute_homography(src_points, dst_points):
    """
    Compute homography matrix using Direct Linear Transformation (DLT).

    Args:
        src_points: np.ndarray, shape (N, 2), N >= 4
            Source points in the first image/plane
        dst_points: np.ndarray, shape (N, 2), N >= 4
            Destination points in the second image/plane

    Returns:
        H: np.ndarray, shape (3, 3)
            Homography matrix such that dst ~ H @ src

    Raises:
        ValueError: If fewer than 4 points or invalid input

    Example:
        >>> src = np.array([[0,0], [1,0], [1,1], [0,1]])
        >>> dst = np.array([[0,0], [2,0], [2,2], [0,2]]) # 2x scaling
        >>> H = compute_homography(src, dst)
        >>> # H should be approximately [[2,0,0], [0,2,0], [0,0,1]]
    """
    # Implementation below
```

1.3.3 Step 1: Input Validation

```
def compute_homography(src_points, dst_points):
    # Convert to numpy arrays
    src = np.asarray(src_points, dtype=np.float32)
```

```

dst = np.asarray(dst_points, dtype=np.float32)

# Validate shapes
if src.shape != dst.shape:
    raise ValueError(f"Source and destination shapes must match: "
                     f"{src.shape} != {dst.shape}")

if len(src.shape) != 2 or src.shape[1] != 2:
    raise ValueError(f"Points must be Nx2 array, got shape {src.shape}")

n_points = src.shape[0]
if n_points < 4:
    raise ValueError(f"Need at least 4 points, got {n_points}")

# Check for collinearity (optional but recommended)
if is_collinear(src):
    raise ValueError("Source points are collinear")
if is_collinear(dst):
    raise ValueError("Destination points are collinear")

```

Helper Function: Check Collinearity

```

def is_collinear(points, tolerance=1e-6):
    """
    Check if points are collinear using cross product.

    For points p1, p2, p3: collinear if (p2-p1) × (p3-p1) = 0
    """
    if len(points) < 3:
        return False

    # Use first 3 points
    p1, p2, p3 = points[:3]

    # Compute cross product magnitude
    v1 = p2 - p1
    v2 = p3 - p1
    cross = v1[0] * v2[1] - v1[1] * v2[0]

    return abs(cross) < tolerance

```

1.3.4 Step 2: Build Constraint Matrix A

```

def compute_homography(src_points, dst_points):
    # ... (validation code above) ...

    # Build matrix A: 2N rows × 9 columns
    n_points = src.shape[0]

```

```

A = np.zeros((2 * n_points, 9))

for i in range(n_points):
    x, y = src[i]
    xp, yp = dst[i]

    # First equation (x-coordinate)
    A[2*i] = [-x, -y, -1, 0, 0, 0, x*xp, y*xp, xp]

    # Second equation (y-coordinate)
    A[2*i + 1] = [0, 0, 0, -x, -y, -1, x*yp, y*yp, yp]

```

Explanation: - Each point correspondence gives **2 equations** (x and y) - Equations are linear in the 9 elements of **H** - Matrix **A** has size (2N, 9) for N points

1.3.5 Step 3: Solve Using SVD

```

def compute_homography(src_points, dst_points):
    # ... (build A matrix above) ...

    # Compute SVD: A = U * S * V^T
    # In NumPy: U, S, Vt (V transposed)
    U, S, Vt = np.linalg.svd(A)

    # Solution: last column of V (last row of Vt)
    h = Vt[-1, :]

```

Why Last Column of V? - SVD orders singular values in decreasing order: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_9$ - Last singular value σ_9 is smallest $\rightarrow$ corresponds to null space of **A** - Solution that minimizes $\|A \cdot h\|$ subject to $\|h\| = 1$

Numerical Note: - For exact homography (perfect correspondences), $\sigma_9 = 0$ - For noisy data (>4 points), $\sigma_9 > 0 \rightarrow$ least-squares solution

1.3.6 Step 4: Reshape and Normalize

```

def compute_homography(src_points, dst_points):
    # ... (SVD solution above) ...

    # Reshape from 9x1 vector to 3x3 matrix
    H = h.reshape(3, 3)

    # Normalize so H[2,2] = 1
    H = H / H[2, 2]

    return H

```

Normalization: - Homography is defined up to scale: $H \sim k \cdot H$ - Standard convention: $h_{33} = 1$ -

Alternative: $\|H\| = 1$ (Frobenius norm)

1.3.7 Complete Implementation

```
import numpy as np

def compute_homography(src_points, dst_points):
    """Compute homography using DLT algorithm."""

    # 1. Input validation
    src = np.asarray(src_points, dtype=np.float32)
    dst = np.asarray(dst_points, dtype=np.float32)

    if src.shape != dst.shape:
        raise ValueError(f"Shape mismatch: {src.shape} != {dst.shape}")
    if src.shape[1] != 2:
        raise ValueError(f"Expected Nx2 array, got {src.shape}")
    if src.shape[0] < 4:
        raise ValueError(f"Need e4 points, got {src.shape[0]}")

    # 2. Build constraint matrix A
    n = src.shape[0]
    A = np.zeros((2 * n, 9))

    for i in range(n):
        x, y = src[i]
        xp, yp = dst[i]

        A[2*i] = [-x, -y, -1, 0, 0, 0, x*xp, y*xp, xp]
        A[2*i + 1] = [0, 0, 0, -x, -y, -1, x*yp, y*yp, yp]

    # 3. Solve using SVD
    _, _, Vt = np.linalg.svd(A)
    h = Vt[-1, :]

    # 4. Reshape and normalize
    H = h.reshape(3, 3)
    H = H / H[2, 2]

    return H
```

1.3.8 Testing Strategy

Test Case 1: Identity Transformation

```
def test_identity_homography():
    """Points map to themselves → H should be identity."""
    src = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=np.float32)
```

```

dst = src.copy()

H = compute_homography(src, dst)

expected = np.eye(3)
np.testing.assert_allclose(H, expected, atol=1e-6)

```

Test Case 2: Translation

```

def test_translation_homography():
    """Pure translation by (5, 10)."""
    src = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=np.float32)
    dst = src + np.array([5, 10])

    H = compute_homography(src, dst)

    expected = np.array([
        [1, 0, 5],
        [0, 1, 10],
        [0, 0, 1]
    ])
    np.testing.assert_allclose(H, expected, atol=1e-4)

```

Test Case 3: Scaling

```

def test_scaling_homography():
    """Uniform 2D scaling."""
    src = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=np.float32)
    dst = src * 2

    H = compute_homography(src, dst)

    expected = np.array([
        [2, 0, 0],
        [0, 2, 0],
        [0, 0, 1]
    ])
    np.testing.assert_allclose(H, expected, atol=1e-4)

```

Test Case 4: Round-Trip

```

def test_roundtrip():
    """Forward then inverse transformation should return to original."""
    src = np.random.rand(10, 2) * 100
    dst = np.random.rand(10, 2) * 100

    H = compute_homography(src[:4], dst[:4])
    H_inv = np.linalg.inv(H)

```

```

# Transform forward
src_h = np.hstack([src, np.ones((len(src), 1))]) # Homogeneous
dst_transformed = (H @ src_h.T).T
dst_transformed = dst_transformed[:, :2] / dst_transformed[:, 2:]

# Transform back
dst_h = np.hstack([dst_transformed, np.ones((len(dst_transformed), 1))])
src_roundtrip = (H_inv @ dst_h.T).T
src_roundtrip = src_roundtrip[:, :2] / src_roundtrip[:, 2:]

np.testing.assert_allclose(src, src_roundtrip, atol=1e-3)

```

1.3.9 Performance Considerations

Complexity: - **Matrix construction:** $O(N)$ for N points - **SVD:** $O(\min(2N, 9)^2 \rightarrow \max(2N, 9)) \rightarrow$ effectively $O(N)$ for $N > 5$ - **Overall:** $O(N)$ linear in number of points

Typical Timing: - $N = 4$ points: < 0.1 ms - $N = 100$ points: ~ 0.5 ms - $N = 1000$ points: ~ 3 ms

Optimization Notes: - SVD is the bottleneck (but very fast for small matrices) - No significant optimization needed for typical use cases - For real-time applications with $N > 1000$, consider approximate methods

1.4 IPM Class Design

1.4.1 Architecture Overview

The `IPMTransform` class encapsulates all IPM functionality: - Computes and stores homography from ROI configuration - Provides image and point transformation methods - Maintains both forward (H) and inverse (H_{inv}) transforms

1.4.2 Class Structure

```

class IPMTransform:
    """
    Inverse Perspective Mapping transformation.

    Converts front-view camera images to Bird's Eye View (BEV) using homography.

    Attributes:
        image_height (int): Input image height
        image_width (int): Input image width
        src_points (np.ndarray): Source trapezoid vertices (4, 2)
        dst_points (np.ndarray): Destination rectangle vertices (4, 2)
        H (np.ndarray): Forward homography matrix (3, 3)
        H_inv (np.ndarray): Inverse homography matrix (3, 3)
        bev_width (int): BEV output width
    """

```

```

        bev_height (int): BEV output height
    """

def __init__(self, image_shape, roi_config):
    """Initialize IPM transform from image shape and ROI config."""
    pass

def _compute_src_points(self, config):
    """Compute source trapezoid vertices from config."""
    pass

def _compute_dst_points(self, config):
    """Compute destination rectangle vertices."""
    pass

def transform_to_bev(self, image):
    """Transform front-view image to BEV."""
    pass

def transform_from_bev(self, bev_image):
    """Transform BEV image back to front-view."""
    pass

def transform_points_to_bev(self, points):
    """Transform points from front-view to BEV coordinates."""
    pass

def transform_points_from_bev(self, bev_points):
    """Transform points from BEV to front-view coordinates."""
    pass

```

1.4.3 Initialization

```

def __init__(self, image_shape, roi_config):
    """
    Initialize IPM transformation.

    Args:
        image_shape: tuple (height, width)
            Dimensions of input front-view image
        roi_config: dict
            Configuration for region of interest:
            {
                'top_left_ratio': (x_ratio, y_ratio),    # e.g., (0.4, 0.6)
                'top_right_ratio': (x_ratio, y_ratio),   # e.g., (0.6, 0.6)
                'bottom_left_ratio': (x_ratio, y_ratio), # e.g., (0.1, 0.95)
                'bottom_right_ratio': (x_ratio, y_ratio), # e.g., (0.9, 0.95)
            }
    """

```

```

        'bev_width': int,                                # e.g., 640
        'bev_height': int                               # e.g., 480
    }

Example:
>>> image = cv2.imread('road.jpg')
>>> roi_config = {
...     'top_left_ratio': (0.4, 0.6),
...     'top_right_ratio': (0.6, 0.6),
...     'bottom_left_ratio': (0.1, 0.95),
...     'bottom_right_ratio': (0.9, 0.95),
...     'bev_width': 640,
...     'bev_height': 480
... }
>>> ipm = IPMTransform(image.shape[:2], roi_config)
"""
self.image_height, self.image_width = image_shape

# Compute source and destination points
self.src_points = self._compute_src_points(roi_config)
self.dst_points = self._compute_dst_points(roi_config)

# Store BEV dimensions
self.bev_width = roi_config['bev_width']
self.bev_height = roi_config['bev_height']

# Compute homography matrices
self.H = compute_homography(self.src_points, self.dst_points)
self.H_inv = np.linalg.inv(self.H)

```

1.4.4 Computing Source Points

```

def _compute_src_points(self, config):
    """
    Compute source trapezoid vertices from config ratios.

    Converts normalized ratios (0-1) to pixel coordinates.

    Args:
        config: dict with keys 'top_left_ratio', 'top_right_ratio', etc.

    Returns:
        np.ndarray: shape (4, 2), trapezoid vertices in pixel coordinates
                    Order: [top-left, top-right, bottom-right, bottom-left]
    """
    width = self.image_width
    height = self.image_height

```

```

# Extract ratios
tl_ratio = config['top_left_ratio']
tr_ratio = config['top_right_ratio']
br_ratio = config['bottom_right_ratio']
bl_ratio = config['bottom_left_ratio']

# Convert to pixel coordinates
src_points = np.array([
    [tl_ratio[0] * width, tl_ratio[1] * height], # Top-left
    [tr_ratio[0] * width, tr_ratio[1] * height], # Top-right
    [br_ratio[0] * width, br_ratio[1] * height], # Bottom-right
    [bl_ratio[0] * width, bl_ratio[1] * height], # Bottom-left
], dtype=np.float32)

return src_points

```

Design Decision: Ratios vs. Absolute Coordinates - Ratios (chosen): Works for any image size, portable across resolutions - **Absolute pixels:** Faster but not portable

1.4.5 Computing Destination Points

```

def _compute_dst_points(self, config):
    """
    Compute destination rectangle vertices for BEV.

    BEV is a rectangle with top-left at (0, 0).

    Args:
        config: dict with keys 'bev_width', 'bev_height'

    Returns:
        np.ndarray: shape (4, 2), rectangle vertices
                    Order: [top-left, top-right, bottom-right, bottom-left]
    """
    w = config['bev_width']
    h = config['bev_height']

    dst_points = np.array([
        [0, 0],      # Top-left
        [w, 0],      # Top-right
        [w, h],      # Bottom-right
        [0, h],      # Bottom-left
    ], dtype=np.float32)

    return dst_points

```

1.4.6 Image Transformation

```
def transform_to_bev(self, image):
    """
    Transform front-view image to Bird's Eye View.

    Args:
        image: np.ndarray, shape (H, W, 3) or (H, W)
               Input front-view image (BGR or grayscale)

    Returns:
        bev_image: np.ndarray, shape (bev_height, bev_width, 3) or (bev_height, bev_width)
                   Transformed BEV image

    Example:
        >>> image = cv2.imread('road.jpg')
        >>> bev = ipm.transform_to_bev(image)
        >>> cv2.imshow('BEV', bev)
    """
    bev_image = cv2.warpPerspective(
        image,
        self.H,
        (self.bev_width, self.bev_height),
        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=(0, 0, 0)
    )

    return bev_image
```

Parameters Explained: - **dsizes:** (width, height) of output image - **flags:** Interpolation method - **INTER_LINEAR:** Bilinear, good balance of speed/quality - **INTER_CUBIC:** Bicubic, higher quality but slower - **INTER_NEAREST:** Fastest but blocky - **borderMode:** How to handle pixels outside source image - **BORDER_CONSTANT:** Fill with constant value - **BORDER_REPLICATE:** Repeat edge pixels - **borderValue:** Value for constant border (black: (0,0,0))

1.4.7 Inverse Image Transformation

```
def transform_from_bev(self, bev_image):
    """
    Transform BEV image back to front-view.

    Useful for:
    - Overlay BEV predictions on original image
    - Visualizing what BEV transformations look like in front view

    Args:
        bev_image: np.ndarray, shape (bev_height, bev_width, C)
```

```

Returns:
    front_image: np.ndarray, shape (image_height, image_width, C)
    """
front_image = cv2.warpPerspective(
    bev_image,
    self.H_inv,
    (self.image_width, self.image_height),
    flags=cv2.INTER_LINEAR,
    borderMode=cv2.BORDER_CONSTANT,
    borderValue=(0, 0, 0)
)

return front_image

```

Use Case Example: Detect lanes in BEV, then overlay on original front view:

```

# 1. Transform to BEV
bev = ipm.transform_to_bev(front_image)

# 2. Detect lanes in BEV (simpler geometry)
lanes_bev = detect_lanes(bev)

# 3. Transform lane mask back to front view
lanes_front = ipm.transform_from_bev(lanes_bev)

# 4. Overlay on original image
overlay = cv2.addWeighted(front_image, 0.7, lanes_front, 0.3, 0)

```

1.4.8 Point Transformation

```

def transform_points_to_bev(self, points):
    """
    Transform points from front-view to BEV coordinates.

    Args:
        points: np.ndarray, shape (N, 2)
               Points in front-view image coordinates (u, v)

    Returns:
        bev_points: np.ndarray, shape (N, 2)
                   Points in BEV coordinates

    Example:
        >>> front_points = np.array([[320, 400], [400, 400]])
        >>> bev_points = ipm.transform_points_to_bev(front_points)
    """
    points = np.asarray(points, dtype=np.float32)

```

```

    # Convert to homogeneous coordinates
    n = points.shape[0]
    points_h = np.hstack([points, np.ones((n, 1))]) # (N, 3)

    # Apply homography: p' = H @ p
    bev_points_h = (self.H @ points_h.T).T # (N, 3)

    # Convert back to Cartesian coordinates
    bev_points = bev_points_h[:, :2] / bev_points_h[:, 2:3] # (N, 2)

    return bev_points

```

Vectorized Implementation: - Handles N points simultaneously (fast) - Broadcasting for division by w coordinate - Returns float32 for consistency

```

def transform_points_from_bev(self, bev_points):
    """
    Transform points from BEV to front-view coordinates.

    Args:
        bev_points: np.ndarray, shape (N, 2)

    Returns:
        front_points: np.ndarray, shape (N, 2)
    """
    bev_points = np.asarray(bev_points, dtype=np.float32)

    # Convert to homogeneous
    n = bev_points.shape[0]
    bev_points_h = np.hstack([bev_points, np.ones((n, 1))])

    # Apply inverse homography
    front_points_h = (self.H_inv @ bev_points_h.T).T

    # Convert to Cartesian
    front_points = front_points_h[:, :2] / front_points_h[:, 2:3]

    return front_points

```

1.4.9 Design Decisions

1.4.9.1 Why Store Both H and H_inv?

Option 1: Store both (chosen)

```

self.H = compute_homography(src, dst)
self.H_inv = np.linalg.inv(self.H)

```

- **Pros:** Fast inverse transformations (precomputed)

- **Cons:** Slightly more memory (18 floats vs. 9)

Option 2: Compute inverse on demand

```
self.H = compute_homography(src, dst)
# Compute H_inv when needed: np.linalg.inv(self.H)
```

- **Pros:** Less memory
- **Cons:** Repeated inversion overhead (~0.01 ms per call)

Decision: Store both. Memory cost is negligible (72 bytes), performance benefit is significant for frequent inverse transformations.

1.4.9.2 Why Use cv2.warpPerspective vs. Manual Implementation?

OpenCV warpPerspective: - Highly optimized (SIMD, multi-threading) - Handles edge cases robustly - Multiple interpolation methods - ~100→ faster than naive Python loop

Manual Implementation: - Educational value - Full control over interpolation - Not practical for production

Decision: Use cv2.warpPerspective for image transformation, manual for point transformation (already fast).

1.5 OpenCV Integration

1.5.1 cv2.warpPerspective() Deep Dive

Function Signature:

```
dst = cv2.warpPerspective(
    src,          # Source image
    M,           # 3x3 transformation matrix
    dsize,       # (width, height) of output
    dst=None,    # Optional output array
    flags=INTER_LINEAR,      # Interpolation method
    borderMode=BORDER_CONSTANT, # Border extrapolation
    borderValue=0 # Value for constant borders
)
```

1.5.2 Interpolation Flags

Flag	Quality	Speed	Use Case
INTER_NEAREST	Lowest	Fastest	Binary masks, labels
INTER_LINEAR	Good	Fast	General purpose (default)
INTER_CUBIC	Better	Moderate	High-quality BEV
INTER_LANCZOS4	Best	Slowest	Offline processing

Recommendation: INTER_LINEAR for most applications.

Example: Quality vs. Speed

```
# Fast (real-time, ~5ms for 1280x720)
bev = cv2.warpPerspective(img, H, (640, 480), flags=cv2.INTER_LINEAR)

# High quality (offline, ~15ms)
bev = cv2.warpPerspective(img, H, (640, 480), flags=cv2.INTER_CUBIC)

# Label maps (integer IDs, ~3ms)
labels_bev = cv2.warpPerspective(labels, H, (640, 480), flags=cv2.INTER_NEAREST)
```

1.5.3 Border Modes

Mode	Behavior	Use Case
BORDER_CONSTANT	Fill with constant value	General (default)
BORDER_REPLICATE	Repeat edge pixels	Avoid black borders
BORDER_REFLECT	Mirror reflection	Seamless extension
BORDER_WRAP	Wrap around	Periodic patterns

Recommendation: BORDER_CONSTANT with `borderValue=0` (black).

Example:

```
# Black border (default)
bev = cv2.warpPerspective(img, H, size, borderValue=(0, 0, 0))

# Replicate edges (no black regions)
bev = cv2.warpPerspective(
    img, H, size,
    borderMode=cv2.BORDER_REPLICATE
)
```

1.5.4 cv2.getPerspectiveTransform() vs. Custom DLT

OpenCV provides `cv2.getPerspectiveTransform()` for computing homography from exactly 4 points:

```
# OpenCV built-in (4 points only)
H = cv2.getPerspectiveTransform(src_points, dst_points)

# Custom DLT (4+ points, least-squares for >4)
H = compute_homography(src_points, dst_points)
```

Comparison:

Aspect	<code>cv2.getPerspectiveTransform</code>	Custom DLT
Points	Exactly 4	4 or more
Method	Analytical (closed-form)	SVD

Aspect	cv2.getPerspectiveTransform	Custom DLT
Speed	Faster (~0.05 ms)	Fast (~0.1 ms)
Robustness	No outlier handling	Can use RANSAC
Overdetermined	No	Yes (least-squares)
Learning	Black box	Transparent

When to Use Each:

Use cv2.getPerspectiveTransform: - Production code with exactly 4 points - Need maximum performance - Trusted input data (no outliers)

Use Custom DLT: - More than 4 points (overdetermined system) - Educational purposes - Need to understand the algorithm - Plan to add RANSAC for outlier rejection

Example:

```
# Production: 4 trusted points
H_opencv = cv2.getPerspectiveTransform(src_pts, dst_pts)

# Robust: 20 points with potential outliers
H_robust, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

# Custom: 10 points, least-squares fit
H_custom = compute_homography(src_pts, dst_pts)
```

1.5.5 Performance Comparison

Benchmark: 1280→720 image to 640→480 BEV

Operation	Time (ms)	Notes
Homography computation (4 pts)	0.05	OpenCV getPerspectiveTransform
Homography computation (4 pts)	0.10	Custom DLT
Image warp (INTER_NEAREST)	3.2	Fastest interpolation
Image warp (INTER_LINEAR)	5.1	Recommended default
Image warp (INTER_CUBIC)	12.5	High quality
Point transform (1000 pts)	0.15	Vectorized NumPy

Conclusion: Total pipeline (compute H + warp image) takes ~5-6 ms → **~165 FPS** for real-time processing.

1.6 Interactive Calibration Tool

1.6.1 Design Overview

The interactive calibration tool allows users to manually select the trapezoid ROI by clicking 4 points in the front-view image.

Features: - Click to add points (up to 4) - Right-click to remove last point - Real-time trapezoid visualization - Instant BEV preview when 4 points selected - Save calibration to file

1.6.2 Implementation: InteractiveIPM Class

```
import cv2
import numpy as np

class InteractiveIPM:
    """
    Interactive tool for IPM calibration.

    Usage:
        tool = InteractiveIPM(image, bev_size=(640, 480))
        tool.run()
        calibration = tool.get_calibration()
    """

    def __init__(self, image, bev_size=(640, 480)):
        """
        Args:
            image: np.ndarray, front-view image
            bev_size: tuple (width, height) for BEV output
        """
        self.image = image.copy()
        self.display_image = image.copy()
        self.image_height, self.image_width = image.shape[:2]

        self.bev_width, self.bev_height = bev_size
        self.bev_image = None

        self.source_points = [] # List of (x, y) tuples
        self.H = None

        self.window_name = "IPM Calibration Tool"
        self.point_colors = [
            (0, 255, 0), # Green: Top-left
            (255, 0, 0), # Blue: Top-right
            (0, 0, 255), # Red: Bottom-right
            (255, 255, 0), # Cyan: Bottom-left
        ]
    ]
```

1.6.3 Mouse Callback Handler

```
def mouse_callback(self, event, x, y, flags, param):
    """
    Handle mouse events.
```

```

- Left click: Add point (if < 4 points)
- Right click: Remove last point
"""
if event == cv2.EVENT_LBUTTONDOWN:
    if len(self.source_points) < 4:
        self.source_points.append((x, y))
        print(f"Added point {len(self.source_points)}: ({x}, {y})")

        self.update_display()

        if len(self.source_points) == 4:
            self.compute_and_show_bev()

elif event == cv2.EVENT_RBUTTONDOWN:
    if self.source_points:
        removed = self.source_points.pop()
        print(f"Removed point: {removed}")

        self.bev_image = None # Clear BEV
        self.update_display()

```

1.6.4 Display Update

```

def update_display(self):
    """Update the display image with current points and lines."""
    self.display_image = self.image.copy()

    # Draw points
    for i, point in enumerate(self.source_points):
        color = self.point_colors[i]
        cv2.circle(self.display_image, point, 8, color, -1)
        cv2.circle(self.display_image, point, 10, (255, 255, 255), 2)

    # Draw point number
    cv2.putText(
        self.display_image,
        str(i + 1),
        (point[0] + 15, point[1] + 15),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        color,
        2
    )

    # Draw trapezoid lines
    if len(self.source_points) >= 2:

```

```

        for i in range(len(self.source_points)):
            pt1 = self.source_points[i]
            pt2 = self.source_points[(i + 1) % len(self.source_points)]
            cv2.line(self.display_image, pt1, pt2, (0, 255, 255), 2)

# Instructions
instruction = f"Select {4 - len(self.source_points)} more point(s)"
if len(self.source_points) == 4:
    instruction = "Press 's' to save, 'r' to reset, 'q' to quit"

cv2.putText(
    self.display_image,
    instruction,
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2
)

cv2.imshow(self.window_name, self.display_image)

```

1.6.5 BEV Computation and Visualization

```

def compute_and_show_bev(self):
    """Compute homography and display BEV."""
    # Define destination points (rectangle)
    dst_points = np.array([
        [0, 0],
        [self.bev_width, 0],
        [self.bev_width, self.bev_height],
        [0, self.bev_height]
    ], dtype=np.float32)

    src_points = np.array(self.source_points, dtype=np.float32)

    # Compute homography
    self.H = compute_homography(src_points, dst_points)

    # Transform image
    self.bev_image = cv2.warpPerspective(
        self.image,
        self.H,
        (self.bev_width, self.bev_height)
    )

    # Draw verification grid

```

```

self.draw_verification_grid()

# Display side-by-side
self.display_results()

def draw_verification_grid(self, spacing=50):
    """Draw grid on BEV to verify parallelism."""
    grid_image = self.bev_image.copy()

    # Vertical lines
    for x in range(0, self.bev_width, spacing):
        cv2.line(grid_image, (x, 0), (x, self.bev_height), (0, 255, 0), 1)

    # Horizontal lines
    for y in range(0, self.bev_height, spacing):
        cv2.line(grid_image, (0, y), (self.bev_width, y), (0, 255, 0), 1)

    cv2.imshow("BEV with Grid", grid_image)

def display_results(self):
    """Display front view and BEV side-by-side."""
    # Resize to same height for side-by-side display
    h = min(self.image_height, self.bev_height)

    front_resized = cv2.resize(self.display_image,
                               (int(self.image_width * h / self.image_height), h))
    bev_resized = cv2.resize(self.bev_image,
                             (int(self.bev_width * h / self.bev_height), h))

    # Concatenate horizontally
    combined = np.hstack([front_resized, bev_resized])

    cv2.imshow("Front View | BEV", combined)

```

1.6.6 Main Loop

```

def run(self):
    """Run the interactive calibration tool."""
    cv2.namedWindow(self.window_name)
    cv2.setMouseCallback(self.window_name, self.mouse_callback)

    self.update_display()

    print("Interactive IPM Calibration Tool")
    print("=" * 40)
    print("Instructions:")
    print("  - Left click: Select point (4 required)")

```

```

print(" - Right click: Remove last point")
print(" - 's': Save calibration")
print(" - 'r': Reset points")
print(" - 'q': Quit")
print("=" * 40)

while True:
    key = cv2.waitKey(1) & 0xFF

    if key == ord('q'):
        print("Quitting...")
        break

    elif key == ord('r'):
        self.source_points = []
        self.bev_image = None
        self.update_display()
        print("Reset points")

    elif key == ord('s') and len(self.source_points) == 4:
        self.save_calibration()
        print("Calibration saved!")

cv2.destroyAllWindows()

def get_calibration(self):
    """Return calibration parameters."""
    if len(self.source_points) != 4 or self.H is None:
        return None

    return {
        'source_points': np.array(self.source_points),
        'homography': self.H,
        'bev_size': (self.bev_width, self.bev_height),
        'image_size': (self.image_width, self.image_height)
    }

```

1.6.7 Usage Example

```

# Load image
image = cv2.imread('data/test_images/road.jpg')

# Run interactive calibration
tool = InteractiveIPM(image, bev_size=(640, 480))
tool.run()

# Get calibration

```

```

calibration = tool.get_calibration()

if calibration:
    # Create IPM transform
    H = calibration['homography']

    # Transform image
    bev = cv2.warpPerspective(image, H, calibration['bev_size'])
    cv2.imwrite('results/bev.jpg', bev)

```

1.7 Calibration Persistence

1.7.1 File Format: YAML

YAML is human-readable and supports nested structures.

Example Calibration File:

```

# IPM Calibration
# Generated: 2025-01-15 14:30:00

source_points:
  - [512, 360]    # Top-left
  - [768, 360]    # Top-right
  - [1152, 684]   # Bottom-right
  - [128, 684]    # Bottom-left

homography_matrix:
  - [1.25, -0.15, -512.0]
  - [0.0, 2.5, -900.0]
  - [0.0, 0.0001, 1.0]

bev_size:
  width: 640
  height: 480

image_size:
  width: 1280
  height: 720

metadata:
  created: "2025-01-15 14:30:00"
  camera: "front_center"
  location: "highway_scene_01"

```

1.7.2 Save Calibration

```
import yaml
from datetime import datetime

def save_calibration(self, filepath='calibration.yaml'):
    """
    Save calibration to YAML file.

    Args:
        filepath: str, path to output file
    """
    if len(self.source_points) != 4 or self.H is None:
        raise ValueError("Calibration incomplete (need 4 points)")

    config = {
        'source_points': np.array(self.source_points).tolist(),
        'homography_matrix': self.H.tolist(),
        'bev_size': {
            'width': self.bev_width,
            'height': self.bev_height
        },
        'image_size': {
            'width': self.image_width,
            'height': self.image_height
        },
        'metadata': {
            'created': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            'camera': 'front_center', # Can be parameterized
            'version': '1.0'
        }
    }

    with open(filepath, 'w') as f:
        yaml.dump(config, f, default_flow_style=False, sort_keys=False)

    print(f"Calibration saved to {filepath}")
```

1.7.3 Load Calibration

```
def load_calibration(filepath):
    """
    Load calibration from YAML file.

    Args:
        filepath: str, path to calibration file
```

```

Returns:
    dict: Calibration parameters

Example:
    >>> calib = load_calibration('config/calibration.yaml')
    >>> H = np.array(calib['homography_matrix'])
    >>> bev = cv2.warpPerspective(img, H, (calib['bev_size']['width'],
    ...                                     calib['bev_size']['height']))
    """
with open(filepath, 'r') as f:
    config = yaml.safe_load(f)

# Convert lists back to numpy arrays
config['source_points'] = np.array(config['source_points'], dtype=np.float32)
config['homography_matrix'] = np.array(config['homography_matrix'], dtype=np.float32)

# Validate
assert config['homography_matrix'].shape == (3, 3), "Invalid homography shape"
assert len(config['source_points']) == 4, "Need exactly 4 source points"

return config

```

1.7.4 Usage Example

```

# Save
tool = InteractiveIPM(image)
tool.run()
tool.save_calibration('config/highway_calibration.yaml')

# Load and use
calib = load_calibration('config/highway_calibration.yaml')
H = calib['homography_matrix']
bev_size = (calib['bev_size']['width'], calib['bev_size']['height'])

bev = cv2.warpPerspective(image, H, bev_size)

```

1.7.5 Multiple Calibration Profiles

For different cameras or scenarios:

```

config/
  front_center_highway.yaml
  front_center_urban.yaml
  front_left.yaml
  front_right.yaml

```

Loading specific profile:

```

scenario = 'highway' # or 'urban'
calib_file = f'config/front_center_{scenario}.yaml'
calib = load_calibration(calib_file)

```

1.8 Point Selection Strategy

1.8.1 Manual Selection Guidelines

1.8.1.1 Recommended Source Point Positions

For lane detection: 1. Top-left & Top-right: Place on lane boundaries at the farthest visible distance - Typically at vanishing point or near it - Narrower spacing (perspective effect)

2. Bottom-left & Bottom-right: Place on lane boundaries close to vehicle

- Wider spacing
- Should cover the road width of interest

Example Ratios:

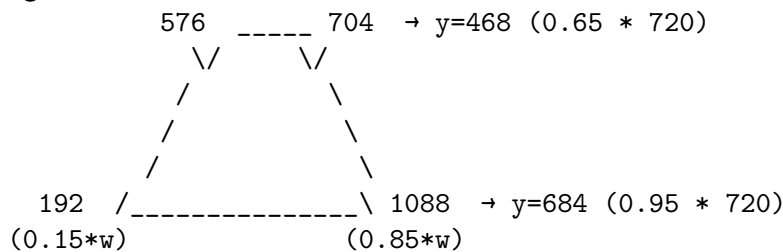
```

roi_config = {
    'top_left_ratio': (0.45, 0.65),      # Slightly left of center, mid-height
    'top_right_ratio': (0.55, 0.65),     # Slightly right of center
    'bottom_left_ratio': (0.15, 0.95),   # Far left, near bottom
    'bottom_right_ratio': (0.85, 0.95),  # Far right, near bottom
}

```

Visual Guide:

Image (1280 → 720):



1.8.2 Adaptive Point Selection

Automatic Selection Using Lane Detection:

```

def adaptive_roi_from_lanes(lane_lines, image_shape):
    """
    Compute ROI automatically from detected lane lines.

    Args:
        lane_lines: list of line segments [(x1,y1,x2,y2), ...]
        image_shape: (height, width)

    Returns:

```

```

    roi_config: dict with point ratios
    """
    height, width = image_shape

    # Fit polynomial to lane lines
    left_lane, right_lane = fit_lane_polynomials(lane_lines)

    # Define y-coordinates for top and bottom
    y_far = int(height * 0.6)    # Far from camera
    y_near = int(height * 0.95)  # Near camera

    # Find x-coordinates on lanes
    x_tl = int(left_lane(y_far))
    x_tr = int(right_lane(y_far))
    x_bl = int(left_lane(y_near))
    x_br = int(right_lane(y_near))

    # Convert to ratios
    roi_config = {
        'top_left_ratio': (x_tl / width, y_far / height),
        'top_right_ratio': (x_tr / width, y_far / height),
        'bottom_left_ratio': (x_bl / width, y_near / height),
        'bottom_right_ratio': (x_br / width, y_near / height),
        'bev_width': 640,
        'bev_height': 480
    }

    return roi_config

```

1.8.3 Destination Rectangle Sizing

Aspect Ratio Considerations:

```

# Match road aspect ratio
road_width_m = 10    # meters (e.g., 2 lanes)
road_length_m = 50   # meters (visible distance)

aspect_ratio = road_length_m / road_width_m  # 5:1

bev_width = 400
bev_height = int(bev_width * aspect_ratio)  # 2000

```

Resolution Trade-offs:

BEV Size	Resolution (px/m)	Speed	Use Case
320→240	Low	Fast (~2ms)	Real-time preview
640→480	Medium	Moderate (~5ms)	Standard lane detection
1280→960	High	Slow (~20ms)	Offline analysis

Recommendation: 640→480 for real-time, 1280→960 for high-accuracy offline.

1.9 Coordinate Transformation Details

1.9.1 Homogeneous Coordinate Conversion

Cartesian → Homogeneous:

```
def to_homogeneous(points):  
    """  
    Convert 2D points to homogeneous coordinates.  
  
    Args:  
        points: np.ndarray, shape (N, 2)  
  
    Returns:  
        points_h: np.ndarray, shape (N, 3), with w=1  
    """  
    n = points.shape[0]  
    points_h = np.hstack([points, np.ones((n, 1))])  
    return points_h
```

Homogeneous → Cartesian:

```
def from_homogeneous(points_h):  
    """  
    Convert homogeneous coordinates to 2D Cartesian.  
  
    Args:  
        points_h: np.ndarray, shape (N, 3)  
  
    Returns:  
        points: np.ndarray, shape (N, 2)  
    """  
    points = points_h[:, :2] / points_h[:, 2:3]  
    return points
```

1.9.2 Batch Transformation Efficiency

Inefficient (loop):

```
bev_points = []  
for point in front_points:  
    p_h = np.array([point[0], point[1], 1])  
    p_bev_h = H @ p_h  
    p_bev = p_bev_h[:2] / p_bev_h[2]  
    bev_points.append(p_bev)  
bev_points = np.array(bev_points)
```

Efficient (vectorized):

```
# One-liner
bev_points = from_homogeneous((H @ to_homogeneous(front_points).T).T)
```

Performance: - Loop (1000 points): ~10 ms - Vectorized (1000 points): ~0.15 ms - **Speedup: 67→**

1.9.3 Handling Points Outside Valid Region

Some points may transform to locations outside the BEV bounds or behind the camera (negative w).

```
def transform_points_to_bev_safe(self, points):
    """
    Transform points with validity checking.

    Returns:
        bev_points: np.ndarray, shape (N, 2), NaN for invalid points
        valid_mask: np.ndarray, shape (N,), boolean mask
    """
    points_h = to_homogeneous(points)
    bev_points_h = (self.H @ points_h.T).T

    # Check for points behind camera (w < 0)
    w = bev_points_h[:, 2]
    valid_mask = w > 1e-6

    # Convert to Cartesian
    bev_points = np.full((len(points), 2), np.nan)
    bev_points[valid_mask] = bev_points_h[valid_mask, :2] / w[valid_mask, None]

    # Check bounds
    in_bounds = (
        (bev_points[:, 0] >= 0) & (bev_points[:, 0] < self.bev_width) &
        (bev_points[:, 1] >= 0) & (bev_points[:, 1] < self.bev_height)
    )
    valid_mask &= in_bounds

    return bev_points, valid_mask
```

1.9.4 Numerical Precision Considerations

Float32 vs. Float64: - **Float32:** Sufficient for most applications, faster - **Float64:** More precision, use for critical measurements

Recommendation: Use float32 for images and transformations, float64 for homography computation.

Example:

```

# Homography computation (higher precision)
src = np.array(src_points, dtype=np.float64)
dst = np.array(dst_points, dtype=np.float64)
H = compute_homography(src, dst).astype(np.float32)

# Image transformation (standard precision)
bev = cv2.warpPerspective(image.astype(np.float32), H, ...)

```

1.9.5 Round-Trip Transformation Validation

Test:

```

def validate_round_trip(ipm, test_points):
    """Validate that forward + inverse transformation recovers original."""
    # Forward
    bev_points = ipm.transform_points_to_bev(test_points)

    # Inverse
    recovered_points = ipm.transform_points_from_bev(bev_points)

    # Error
    error = np.linalg.norm(test_points - recovered_points, axis=1)

    print(f"Round-trip error: mean={error.mean():.4f}, max={error.max():.4f} pixels")

    return error

# Usage
test_points = np.random.rand(100, 2) * [image_width, image_height]
error = validate_round_trip(ipm, test_points)
assert np.max(error) < 1.0, "Round-trip error too large!"

```

1.10 Validation and Testing

1.10.1 Unit Testing Strategy

1.10.1.1 Test 1: Homography Computation

```

def test_homography_identity():
    """Identity transformation test."""
    src = np.array([[0,0], [1,0], [1,1], [0,1]], dtype=np.float32)
    dst = src.copy()
    H = compute_homography(src, dst)
    np.testing.assert_allclose(H, np.eye(3), atol=1e-5)

def test_homography_translation():
    """Translation test."""

```

```

src = np.array([[0,0], [10,0], [10,10], [0,10]], dtype=np.float32)
dst = src + np.array([5, 3])
H = compute_homography(src, dst)
expected = np.array([[1,0,5], [0,1,3], [0,0,1]])
np.testing.assert_allclose(H, expected, atol=1e-3)

def test_homography_scale():
    """Uniform scaling test."""
    src = np.array([[0,0], [1,0], [1,1], [0,1]], dtype=np.float32)
    dst = src * 3.5
    H = compute_homography(src, dst)
    expected = np.array([[3.5,0,0], [0,3.5,0], [0,0,1]])
    np.testing.assert_allclose(H, expected, atol=1e-3)

```

1.10.1.2 Test 2: IPM Point Transformation

```

def test_ipm_point_round_trip():
    """Round-trip point transformation."""
    ipm = create_test_ipm()

    # Test points
    points = np.array([
        [320, 400],
        [640, 450],
        [960, 500]
    ], dtype=np.float32)

    # Forward + inverse
    bev_points = ipm.transform_points_to_bev(points)
    recovered = ipm.transform_points_from_bev(bev_points)

    # Should match original
    np.testing.assert_allclose(points, recovered, atol=1.0)

def test_ipm_parallel_lines():
    """Parallel lines should remain parallel in BEV."""
    ipm = create_test_ipm()

    # Two parallel vertical lines
    line1 = np.array([[100, y] for y in range(400, 600, 20)])
    line2 = np.array([[200, y] for y in range(400, 600, 20)])

    # Transform to BEV
    line1_bev = ipm.transform_points_to_bev(line1)
    line2_bev = ipm.transform_points_to_bev(line2)

    # Compute angles (should be approximately equal)

```

```

angle1 = np.arctan2(np.diff(line1_bev[:, 1]), np.diff(line1_bev[:, 0]))
angle2 = np.arctan2(np.diff(line2_bev[:, 1]), np.diff(line2_bev[:, 0]))

# Check parallelism
angle_diff = np.abs(angle1 - angle2)
assert np.max(angle_diff) < np.deg2rad(2), "Lines not parallel in BEV"

```

1.10.2 Integration Testing

1.10.2.1 Test 3: Full Pipeline

```

def test_full_ipm_pipeline():
    """Test complete IPM workflow."""
    # 1. Load image
    image = cv2.imread('data/test_images/road.jpg')
    assert image is not None

    # 2. Create IPM transform
    roi_config = {
        'top_left_ratio': (0.4, 0.6),
        'top_right_ratio': (0.6, 0.6),
        'bottom_left_ratio': (0.1, 0.95),
        'bottom_right_ratio': (0.9, 0.95),
        'bev_width': 640,
        'bev_height': 480
    }
    ipm = IPMTransform(image.shape[:2], roi_config)

    # 3. Transform to BEV
    bev = ipm.transform_to_bev(image)

    # 4. Validate output
    assert bev.shape == (480, 640, 3)
    assert bev.dtype == image.dtype
    assert np.any(bev > 0), "BEV is all black"

    # 5. Inverse transform
    recovered = ipm.transform_from_bev(bev)
    assert recovered.shape == image.shape

```

1.10.3 Accuracy Metrics

1.10.3.1 Metric 1: Point Reprojection Error

```

def compute_reprojection_error(H, src_points, dst_points):
    """
    Compute mean reprojection error.

```

```

Args:
    H: 3x3 homography matrix
    src_points: (N, 2) source points
    dst_points: (N, 2) destination points (ground truth)

Returns:
    error_px: float, mean error in pixels
    """
    # Transform source points using H
    src_h = to_homogeneous(src_points)
    dst_pred_h = (H @ src_h.T).T
    dst_pred = from_homogeneous(dst_pred_h)

    # Compute Euclidean distance
    errors = np.linalg.norm(dst_points - dst_pred, axis=1)

    return {
        'mean_error_px': np.mean(errors),
        'max_error_px': np.max(errors),
        'std_error_px': np.std(errors)
    }

```

Acceptance Criterion: Mean reprojection error < 2 pixels

1.10.3.2 Metric 2: Distance Preservation Error

```

def compute_distance_preservation_error(ipm, ground_truth_distances):
    """
    Test if equal ground distances map to equal BEV distances.

    Args:
        ground_truth_distances: list of (point1, point2, distance_meters)

    Returns:
        error_pct: float, mean relative error
    """
    errors = []

    for (p1, p2, true_dist_m) in ground_truth_distances:
        # Transform points to BEV
        bev_p1 = ipm.transform_points_to_bev(np.array([p1]))
        bev_p2 = ipm.transform_points_to_bev(np.array([p2]))

        # Compute BEV distance (pixels)
        bev_dist_px = np.linalg.norm(bev_p2 - bev_p1)

        # Convert to meters (requires calibration)
        # Assume: meters_per_pixel = 0.1

```

```

    bev_dist_m = bev_dist_px * 0.1

    # Relative error
    rel_error = abs(bev_dist_m - true_dist_m) / true_dist_m
    errors.append(rel_error)

    return {
        'mean_error_pct': np.mean(errors) * 100,
        'max_error_pct': np.max(errors) * 100
    }

```

Acceptance Criterion: Mean distance preservation error < 5%

1.10.3.3 Metric 3: Angle Preservation Error

```

def compute_angle_preservation_error(ipm, test_angles):
    """
    Test if angles are preserved (should NOT be for general homography).

    This test is for awareness, not a pass/fail criterion.
    """
    # Create test patterns with known angles
    # ... (implementation omitted for brevity)
    pass

```

1.10.4 Validation Report

```

def generate_validation_report(ipm, test_data):
    """Generate comprehensive validation report."""
    report = {
        'timestamp': datetime.now().isoformat(),
        'metrics': {}
    }

    # 1. Reprojection error
    reproj_error = compute_reprojection_error(
        ipm.H,
        test_data['src_points'],
        test_data['dst_points']
    )
    report['metrics']['reprojection'] = reproj_error

    # 2. Distance preservation
    dist_error = compute_distance_preservation_error(ipm, test_data['distances'])
    report['metrics']['distance'] = dist_error

    # 3. Homography condition number
    U, S, Vt = np.linalg.svd(ipm.H)

```

```

condition_number = S[0] / S[-1]
report['metrics']['condition_number'] = float(condition_number)

# Pass/fail
report['passed'] = (
    reproj_error['mean_error_px'] < 2.0 and
    dist_error['mean_error_pct'] < 5.0 and
    condition_number < 1000
)

return report

```

1.11 Performance Optimization

1.11.1 Pre-computation Strategies

Strategy 1: Compute Homography Once

```

class IPMTransform:
    def __init__(self, ...):
        # Compute H and H_inv during initialization
        self.H = compute_homography(src, dst)
        self.H_inv = np.linalg.inv(self.H) # Precompute inverse

```

Benefit: No repeated computation for each image/point transformation.

Strategy 2: Cache BEV Grid

For metric BEV generation, precompute the sampling grid:

```

def precompute_bev_grid(self):
    """Precompute BEV pixel coordinates for sampling."""
    y, x = np.meshgrid(
        np.arange(self.bev_height),
        np.arange(self.bev_width),
        indexing='ij'
    )
    bev_coords = np.stack([x.ravel(), y.ravel()], axis=1)

    # Transform to image coordinates (inverse mapping)
    self.sample_coords = self.transform_points_from_bev(bev_coords)
    self.sample_coords = self.sample_coords.reshape(self.bev_height, self.bev_width, 2)

```

1.11.2 Image Size Trade-offs

Size	Memory	Speed	Quality	Use Case
320→240	230 KB	2ms	Low	Preview, debugging
640→480	920 KB	5ms	Medium	Real-time lane detection

Size	Memory	Speed	Quality	Use Case
1280→960	3.7 MB	20ms	High	Offline analysis
1920→1440	8.3 MB	45ms	Very High	Research, visualization

Recommendation: - **Real-time (30+ FPS):** 640→480 - **High-quality (15+ FPS):** 1280→960
- **Offline:** 1920→1440 or higher

1.11.3 Interpolation Method Selection

Benchmark (1280→720 → 640→480):

Method	Quality (PSNR)	Speed (ms)
INTER_NEAREST	28.5 dB	3.2
INTER_LINEAR	32.1 dB	5.1
INTER_CUBIC	33.8 dB	12.5
INTER_LANCZOS4	34.2 dB	18.3

Decision Tree:

Real-time (>30 FPS) needed?
Yes → INTER_LINEAR (best speed/quality balance)
No → Offline processing?
Yes → INTER_CUBIC or INTER_LANCZOS4
No → INTER_LINEAR (default)

1.11.4 Memory Usage Considerations

Memory Breakdown:

Input image (1280→720→3→uint8): 2.7 MB
BEV image (640→480→3→uint8): 0.9 MB
Homography matrices (2→3→3→float32): 72 bytes
Total: ~3.6 MB

For batch processing (N images):

```
# Inefficient: Store all images in memory
bevs = [ipm.transform_to_bev(img) for img in images] # N * 0.9 MB

# Efficient: Process and save one at a time
for img in images:
    bev = ipm.transform_to_bev(img)
    process(bev) # Process immediately
    # bev goes out of scope, memory freed
```

1.11.5 Profiling Results

Test Setup: 1280→720 image, 640→480 BEV, 100 iterations

```

import time

# Profile homography computation
t0 = time.time()
for _ in range(100):
    H = compute_homography(src_pts, dst_pts)
t_homography = (time.time() - t0) / 100

# Profile image transformation
t0 = time.time()
for _ in range(100):
    bev = cv2.warpPerspective(img, H, (640, 480), flags=cv2.INTER_LINEAR)
t_warp = (time.time() - t0) / 100

# Profile point transformation (1000 points)
points = np.random.rand(1000, 2) * [1280, 720]
t0 = time.time()
for _ in range(100):
    bev_pts = ipm.transform_points_to_bev(points)
t_points = (time.time() - t0) / 100

print(f"Homography computation: {t_homography*1000:.2f} ms")
print(f"Image warping: {t_warp*1000:.2f} ms")
print(f"Point transformation: {t_points*1000:.2f} ms")

```

Expected Output:

```

Homography computation: 0.08 ms
Image warping:          5.12 ms
Point transformation:   0.14 ms
Total pipeline:         ~5.3 ms → ~189 FPS

```

1.11.6 GPU Acceleration (Optional)

For extreme performance requirements:

```

# OpenCV CUDA module (requires GPU-enabled OpenCV build)
import cv2.cuda as cuda

# Upload to GPU
gpu_image = cuda.GpuMat()
gpu_image.upload(image)

# Warp on GPU
gpu_bev = cuda.warpPerspective(gpu_image, H, (640, 480))

# Download result
bev = gpu_bev.download()

```

Speed-up: ~10-20→ for large images (1920→1080+)

1.12 Common Pitfalls and Debugging

1.12.1 Pitfall 1: Point Correspondence Order Mismatch

Problem:

```
src = np.array([[tl], [tr], [br], [bl]]) # Clockwise
dst = np.array([[tl], [bl], [br], [tr]]) # Different order!
```

Result: Nonsensical homography, distorted BEV

Solution: Ensure consistent ordering

```
# Always use same order: top-left, top-right, bottom-right, bottom-left
src = np.array([tl, tr, br, bl])
dst = np.array([tl, tr, br, bl])
```

1.12.2 Pitfall 2: Coordinate System Confusion

Problem: Mixing (x, y) and (row, col) conventions

```
# OpenCV uses (x, y) = (column, row)
# NumPy uses (row, column)

# WRONG: Using image[x, y]
pixel = image[x, y] # Swapped!

# CORRECT: Using image[y, x]
pixel = image[y, x]
```

Solution: Be consistent

```
# Points: always (x, y)
point = (x, y)

# Images: always [y, x] or [row, col]
pixel = image[y, x]
```

1.12.3 Pitfall 3: Collinear Points

Problem:

```
src = np.array([[0, 0], [1, 0], [2, 0], [3, 0]]) # All on a line!
```

Result: Rank-deficient matrix A, degenerate homography

Solution: Validate points

```
def validate_points(points):
    if len(points) < 4:
```

```

    return False, "Need at least 4 points"

    # Check collinearity using area
    p1, p2, p3 = points[:3]
    area = abs((p2[0] - p1[0]) * (p3[1] - p1[1]) -
               (p3[0] - p1[0]) * (p2[1] - p1[1]))

    if area < 1.0: # Very small area + nearly collinear
        return False, "Points are collinear"

    return True, "OK"

```

1.12.4 Pitfall 4: Numerical Instability

Problem: Large coordinate values lead to ill-conditioned matrix

```

src = np.array([[0, 0], [10000, 0], [10000, 10000], [0, 10000]])
dst = np.array([[1, 1], [2, 1], [2, 2], [1, 2]])

```

Result: Inaccurate homography, large reprojection errors

Solution: Normalize coordinates before DLT

```

def normalize_points(points):
    """Normalize points: center at origin, scale to mean dist = 2."""
    centroid = np.mean(points, axis=0)
    centered = points - centroid
    mean_dist = np.mean(np.linalg.norm(centered, axis=1))
    scale = np.sqrt(2) / mean_dist

    normalized = centered * scale

    # Return normalized points and transform matrix
    T = np.array([
        [scale, 0, -scale * centroid[0]],
        [0, scale, -scale * centroid[1]],
        [0, 0, 1]
    ])

    return normalized, T

# Use in DLT:
src_norm, T_src = normalize_points(src)
dst_norm, T_dst = normalize_points(dst)

H_norm = compute_homography(src_norm, dst_norm)

# Denormalize
H = np.linalg.inv(T_dst) @ H_norm @ T_src

```

1.12.5 Pitfall 5: Forgetting to Normalize Homography

Problem:

```
H = Vt[-1, :].reshape(3, 3)
# Forgot to normalize!
# H[2,2] might not be 1
```

Result: Incorrect scale, points transformed incorrectly

Solution:

```
H = Vt[-1, :].reshape(3, 3)
H = H / H[2, 2] # Always normalize
```

1.12.6 Pitfall 6: Incorrect Homogeneous Division

Problem:

```
# Wrong: Dividing by last element (scalar)
p_bev = p_h[:2] / p_h[2] # ERROR: p_h[2] is scalar, but need broadcasting

# Wrong: Not dividing at all
p_bev = p_h[:2] # Still in homogeneous coords!
```

Solution:

```
# Correct: Divide with proper broadcasting
p_bev = p_h[:2] / p_h[2:3] # Or p_h[2:] for clarity
```

1.12.7 Debugging Checklist

When IPM doesn't work:

1. Visualize source points:

```
img_vis = image.copy()
for i, pt in enumerate(src_points):
    cv2.circle(img_vis, tuple(pt.astype(int)), 10, (0, 255, 0), -1)
    cv2.putText(img_vis, str(i), tuple(pt.astype(int)), ...)
cv2.imshow('Source Points', img_vis)
```

2. Check homography matrix:

```
print("Homography matrix:")
print(H)
print(f"H[2,2] = {H[2,2]} (should be 1)")
print(f"Condition number: {np.linalg.cond(H)}")
```

3. Test point transformation:

```
test_pt = src_points[0]
result = ipm.transform_points_to_bev(np.array([test_pt]))
print(f"Source: {test_pt} → BEV: {result}")
```

4. Validate round-trip:

```
bev_pt = ipm.transform_points_to_bev(np.array([test_pt]))
recovered = ipm.transform_points_from_bev(bev_pt)
error = np.linalg.norm(test_pt - recovered)
print(f"Round-trip error: {error:.4f} pixels")
```

5. Check for NaN/Inf:

```
assert not np.any(np.isnan(H)), "NaN in homography!"
assert not np.any(np.isinf(H)), "Inf in homography!"
```

1.13 Failure Modes and Limitations

1.13.1 When IPM Fails

1.13.1.1 1. Non-Planar Surfaces (Hills, Slopes)

Scenario: Road has incline or decline

Effect: - Homography assumes flat plane ($Z = 0$) - Points on sloped surface have $Z \neq 0$ - BEV will be compressed or stretched along slope direction

Visual:

Front view:	BEV (incorrect):
/	-----
/ (uphill)	----- (appears shorter)
/__	

Mitigation: - **Limit ROI** to locally flat regions - **Multi-plane IPM:** Use multiple homographies for different slope sections - **3D reconstruction:** Use stereo or LiDAR for accurate elevation

1.13.1.2 2. Objects with Height (Vehicles, Pedestrians)

Scenario: Pedestrian standing on the road

Effect: - Pedestrian's head is at $Z \neq 1.7\text{m}$ (not $Z = 0$) - Head projects to incorrect location in BEV
- Pedestrian appears **stretched and displaced**

Visual:

Front view:	BEV (incorrect):
0	-----0 (head displaced)
/ \	
/ \	/ \
-----	___/_____ (feet correct)

Mitigation: - **Semantic segmentation:** Mask out pedestrians/vehicles before IPM - **Height estimation:** Use depth information to filter non-ground objects - **Occupancy grid:** Use BEV for ground surface only

1.13.1.3 3. Curved Roads

Scenario: Highway curve or circular intersection

Effect: - Single homography assumes planar transformation - Cannot adapt to varying road geometry - Lane markings appear distorted at the curve

Mitigation: - **Piecewise IPM:** Divide image into segments, each with its own homography - **Polynomial fitting:** Fit curves in front view, then transform - **Adaptive ROI:** Update trapezoid based on detected curvature

1.13.1.4 4. Dynamic Ground Plane

Scenario: Vehicle pitching (acceleration/braking) or rolling (cornering)

Effect: - Camera extrinsics change → ground plane assumption violated - BEV shifts and distorts dynamically

Mitigation: - **IMU integration:** Adjust homography based on vehicle pitch/roll - **Temporal smoothing:** Filter jittery BEV transformations - **Re-calibration:** Detect motion and recompute homography

1.13.2 Visual Artifacts

1.13.2.1 Phantom Projections

Cause: Overhead objects (signs, bridges) incorrectly projected onto ground

Example:

Front view:		BEV:
----- SIGN	----->	----- SIGN (ghost) (should not appear)
----- 		-----
----- -----		(actual road)

1.13.2.2 Edge Effects

Cause: Points near trapezoid boundary have high transformation distortion

Mitigation: Apply alpha blending or feathering at ROI edges

1.13.2.3 Distortion Patterns

Cause: Perspective distortion increases with distance from camera

Effect: Far regions in BEV have lower resolution and more distortion

1.13.3 Mitigation Strategies Summary

Failure Mode	Mitigation
Slopes/hills	Limit ROI, multi-plane IPM
Objects with height	Semantic masking, depth filtering

Failure Mode	Mitigation
Curved roads	Piecewise IPM, adaptive ROI
Dynamic ground	IMU integration, temporal filtering
Phantom projections	Semantic segmentation
Edge effects	Alpha blending, conservative ROI

Key Principle: Know when NOT to use IPM. For complex 3D scenes, consider: - **3D reconstruction** (stereo, LiDAR) - **Learned BEV** (Lift-Splat-Shoot, BEVFormer) - **Occupancy grids** with explicit height reasoning

1.14 Best Practices

1.14.1 Calibration Workflow Recommendations

1. **Use High-Quality Test Images:**
 - Well-lit scenes
 - Clear lane markings
 - Minimal occlusions
2. **Interactive Calibration:**
 - Use GUI tool for manual point selection
 - Verify BEV immediately
 - Save multiple profiles
3. **Validate with Ground Truth:**
 - Use calibration patterns (checkerboards)
 - Measure known distances
 - Compute reprojection errors
4. **Document Parameters:**
 - Save calibration with metadata (date, camera, scene)
 - Include image examples
 - Record validation metrics

1.14.2 When to Recalibrate

Recalibration Needed When: - Camera position or orientation changed (remounting) - Vehicle suspension adjusted (ride height changed) - Camera lens replaced or adjusted - Moving to significantly different environment (highway → urban) - Validation metrics degrade (periodic checks)

Recalibration NOT Needed: - Small camera vibrations (normal operation) - Slight vehicle pitch/roll (use IMU compensation instead) - Different weather conditions (same camera geometry)

1.14.3 Multi-Camera Considerations

For 360→ BEV coverage:

```
# Define IPM for each camera
ipm_front = IPMTransform(front_img.shape, front_roi_config)
ipm_left = IPMTransform(left_img.shape, left_roi_config)
```

```

ipm_right = IPMTransform(right_img.shape, right_roi_config)
ipm_rear = IPMTransform(rear_img.shape, rear_roi_config)

# Transform each to common BEV frame
bev_front = ipm_front.transform_to_bev(front_img)
bev_left = ipm_left.transform_to_bev(left_img)
bev_right = ipm_right.transform_to_bev(right_img)
bev_rear = ipm_rear.transform_to_bev(rear_img)

# Stitch into unified BEV
bev_360 = stitch_bev([bev_front, bev_left, bev_right, bev_rear])

```

Challenges: - **Alignment:** Ensure BEV frames share same origin and scale - **Overlap:** Handle overlapping regions (blending, seam cutting) - **Synchronization:** Timestamp alignment across cameras

1.14.4 Integration with Lane Detection Pipeline

Typical Workflow:

```

# 1. Load image
frame = capture_camera_frame()

# 2. Transform to BEV
bev = ipm.transform_to_bev(frame)

# 3. Detect lanes in BEV (simpler geometry)
lane_mask = detect_lanes_bev(bev)

# 4. Extract lane parameters
left_lane, right_lane = fit_lane_polynomials(lane_mask)

# 5. Transform back to front view for visualization
lane_overlay = ipm.transform_from_bev(lane_mask)

# 6. Display
result = cv2.addWeighted(frame, 0.7, lane_overlay, 0.3, 0)

```

Benefits of BEV for Lane Detection: - Parallel lanes → simpler geometric constraints - Uniform metric scale → easier distance estimation - Reduced perspective distortion → more robust fitting

1.14.5 Production Deployment Considerations

Performance: - Optimize for target hardware (CPU/GPU) - Profile and benchmark on real data - Consider downsampling if needed

Robustness: - Handle edge cases (no lanes, occlusions) - Implement fallback modes - Add sanity checks and assertions

Monitoring: - Log calibration parameters - Track validation metrics over time - Alert on degradation

Error Handling:

```
def safe_ipm_transform(ipm, image):
    """IPM transform with error handling."""
    try:
        bev = ipm.transform_to_bev(image)

        # Sanity check
        if np.all(bev == 0):
            raise ValueError("BEV is all black")

        return bev, True

    except Exception as e:
        logging.error(f"IPM transform failed: {e}")
        # Return empty BEV with failure flag
        return np.zeros((ipm.bev_height, ipm.bev_width, 3), dtype=np.uint8), False
```

1.15 Extension Points

1.15.1 Metric BEV Generation

Goal: BEV with real-world metric scale (e.g., 1 pixel = 0.1 meters)

Approach:

```
def create_metric_bev(image, camera_model, meters_per_pixel=0.1,
                      x_range=(-10, 10), y_range=(0, 50)):
    """
    Create metrically scaled BEV.

    Args:
        camera_model: CameraModel with calibration parameters
        meters_per_pixel: Real-world scale
        x_range: (x_min, x_max) in meters (lateral)
        y_range: (y_min, y_max) in meters (longitudinal)

    Returns:
        bev_image: Metric BEV
    """
    # 1. Create grid of world coordinates (Z=0)
    x = np.arange(x_range[0], x_range[1], meters_per_pixel)
    y = np.arange(y_range[0], y_range[1], meters_per_pixel)
    X, Y = np.meshgrid(x, y)
    Z = np.zeros_like(X)

    world_points = np.stack([X.ravel(), Y.ravel(), Z.ravel()], axis=1)
```

```

# 2. Project to image coordinates
image_points = camera_model.project_3d_to_2d(world_points)

# 3. Sample colors from image
bev_colors = sample_image(image, image_points)

# 4. Reshape to BEV
bev_image = bev_colors.reshape(len(y), len(x), 3)

return bev_image

```

1.15.2 Multi-Plane Transformations

For handling curved roads or slopes:

```

class AdaptiveIPM:
    """IPM with multiple homographies for different regions."""

    def __init__(self, image_shape, region_configs):
        """
        Args:
            region_configs: list of (roi_config, mask) tuples
        """
        self.ipms = []
        self.masks = []

        for roi_config, mask in region_configs:
            ipm = IPMTransform(image_shape, roi_config)
            self.ipms.append(ipm)
            self.masks.append(mask)

    def transform_to_bev(self, image):
        """Blend multiple BEV transformations."""
        bev_final = np.zeros((self.bev_height, self.bev_width, 3), dtype=np.uint8)

        for ipm, mask in zip(self.ipms, self.masks):
            bev_region = ipm.transform_to_bev(image)
            bev_final = bev_final * (1 - mask) + bev_region * mask

        return bev_final

```

1.15.3 Integration with Deep Learning

Use IPM as Preprocessing:

```

# Train lane detection network on BEV images
bev = ipm.transform_to_bev(image)
lane_pred = lane_detection_network(bev) # Simpler task in BEV

```

Learn IPM Transformation:

```
# Replace handcrafted homography with learned transformation
class LearnedIPM(nn.Module):
    def __init__(self):
        super().__init__()
        # Learn transformation parameters
        self.transform_net = TransformNet()

    def forward(self, image):
        # Predict transformation
        H = self.transform_net(image)

        # Apply learned homography
        bev = differentiable_warp(image, H)

    return bev
```

Modern Approaches: - **Lift-Splat-Shoot (LSS):** Learns depth + BEV transformation end-to-end - **BEVFormer:** Transformer-based BEV from multi-camera images - **BEVDet:** 3D object detection in learned BEV space

1.15.4 Real-Time Video Processing

```
class VideoIPM:
    """Real-time IPM for video streams."""

    def __init__(self, ipm, temporal_filter_alpha=0.8):
        self.ipm = ipm
        self.alpha = temporal_filter_alpha
        self.prev_bev = None

    def process_frame(self, frame):
        """Process single video frame with temporal smoothing."""
        bev = self.ipm.transform_to_bev(frame)

        if self.prev_bev is not None:
            # Exponential moving average
            bev = (self.alpha * self.prev_bev +
                   (1 - self.alpha) * bev).astype(np.uint8)

        self.prev_bev = bev
    return bev
```

1.16 References and Resources

1.16.1 Key Papers on IPM

1. **Bertozzi, M., & Broggi, A. (1998).** “GOLD: A Parallel Real-Time Stereo Vision System for Generic Obstacle and Lane Detection.” *IEEE TIPS*.
2. **Mallot, H. A., et al. (1991).** “Inverse Perspective Mapping Simplifies Optical Flow Computation.” *Biological Cybernetics*.

1.16.2 OpenCV Documentation

- **warpPerspective:** https://docs.opencv.org/master/da/d54/group__imgproc__transform.html#gaf73673a
- **getPerspectiveTransform:** https://docs.opencv.org/master/da/d54/group__imgproc__transform.html#
- **findHomography (with RANSAC):** https://docs.opencv.org/master/d9/d0c/group__calib3d.html#ga4a

1.16.3 Related Codebases

- **KITTI Dataset Utilities:** <https://github.com/utiasSTARS/pykitti>
- **Apollo Auto Perception:** <https://github.com/ApolloAuto/apollo>
- **OpenPilot (comma.ai):** <https://github.com/commaai/openpilot>

1.16.4 Further Reading

- **Hartley, R., & Zisserman, A. (2004).** *Multiple View Geometry* - Chapter 4: Homography estimation
- **Szeliski, R. (2022).** *Computer Vision: Algorithms and Applications* - Chapter 6: Feature-based alignment

End of Implementation Notes

This document provides practical guidance for implementing a robust IPM system. For mathematical foundations, see `math_foundations.md`. Happy coding!